LibriCore — Library Management System

Frontend Architecture & REST API Integration Manual

Frontend: React 19 + Vite 8 **Backend:** Spring Boot 4.1 + Java 21 **Proxy Port:** 5173 → 8080 **Date:** October 2026

---

1. Architectural Overview & Network Flow

The LibriCore client is built as a single-page application using React 19 and Vite. The backend is an existing Spring Boot REST service running on port 8080. Because the backend does not configure Cross-Origin Resource Sharing (CORS), direct browser calls to port 8080 would be blocked. To connect seamlessly **without altering any backend Java code**, a reverse proxy was implemented in Vite:

```
Browser (React UI)
  │ (User actions on DashboardPage, BooksPage, MembersPage)
  ▼
Service Layer (bookService.js & memberService.js)
  │ (Field normalization, backend param adaptation, validation)
  ▼
API Client (api/client.js)
  │ (Fetch wrapper, JSON serialization, console error logger)
  ▼
Vite Dev Proxy (Port 5173)
  │ Maps: /api/*  ──►  http://localhost:8080/api/* (Zero CORS errors)
  ▼
Spring Boot REST Controllers (Port 8080)
  (BookController.java & MemberController.java)
```

> **Vite Reverse Proxy Configuration (vite.config.js)**
> Requests sent to `/api/*` on the local dev server are forwarded by Vite to `http://localhost:8080/api/*`. The browser interacts only with same-origin port 5173, completely bypassing browser CORS restrictions.

2. Critical Backend Nuances & Frontend Normalization

- **Field Inversion in `BookService.toResponse()`:**
  In `BookService.java` (lines 119-126), the constructor arguments for `totalCopies` and `availableCopies` are passed swapped. The frontend service layer (`bookService.normalizeBook()`) automatically restores semantic accuracy so the UI always reflects true stock availability.
- **Sort Parameter declarations in `MemberController.java`:**
  The member controller declares `@RequestParam String shortBy` (spelled with an 'h'). The frontend adapter `memberService.js` transparently maps standard `sortBy` calls to `shortBy`.
- **Error Diagnostics:**
  All network timeouts and HTTP errors (400, 404, 409, 500) are automatically formatted and printed to `console.error` with full payload context for real-time debugging.

3. Book Management APIs (BookController.java)

`GET` `/api/books` `PAGE<BOOKRESPONSE>`

Purpose: Fetches paginated book catalog records with sorting and search filtering.

PARAMETER	TYPE	DESCRIPTION
`page`	Integer (default 0)	Zero-indexed page number
`size`	Integer (default 10)	Number of records per page (5, 10, 20, 50)
`sortBy`	String (default "bookId")	Column: `bookId`, `title`, `author`, `isbn`
`direction`	String (default "asc")	Sort order: `asc` or `desc`
`search`	String (optional)	Filters by book title or author (case-insensitive)

Frontend Function: `bookService.getBooks({ page, size, sortBy, direction, search })` in `src/api/bookService.js`

UI Component: `BooksPage.jsx` (data table & pagination) and `DashboardPage.jsx` (KPI metrics & highlights).

`GET` `/api/books/{id}` `BOOKRESPONSE`

Purpose: Retrieves detailed metadata for a single catalog record by ID.

Frontend Function: `bookService.getBookById(id)`

`POST` `/api/books` `STATUS: 201 CREATED`

Purpose: Creates and registers a new book title in the library database.

Request Body (BookRequest):

```
{
  "title": "Clean Code",
  "author": "Robert C. Martin",
  "isbn": "978-0132350884",
  "totalCopies": 5
}
```

Validation & Error Handling: Returns `409 Conflict` if the ISBN already exists. The frontend captures this and renders an inline warning banner.

Frontend Function: `bookService.createBook(bookData)` invoked from `BookFormModal.jsx`.

`PUT` `/api/books/{id}` `STATUS: 200 OK`

Purpose: Updates an existing book title, author, ISBN, or stock count.

Frontend Function: `bookService.updateBook(id, bookData)` invoked from `BookFormModal.jsx`.

`DELETE` `/api/books/{id}` `STATUS: 204 NO CONTENT`

Purpose: Permanently deletes a book from the catalog.

Frontend Function: `bookService.deleteBook(id)` invoked from `DeleteBookModal.jsx`.

---

4. Member Directory APIs (MemberController.java)

`GET` **`/api/members`** `PAGE<MEMBERRESPONSE>`

Purpose: Fetches patron records filtered by account status (`ACTIVE` / `INACTIVE`) or search query.

PARAMETER	TYPE	DESCRIPTION
`page`	Integer (default 0)	Zero-indexed page number
`size`	Integer (default 10)	Records per page
`shortBy`	String (default "memberId")	Declared as `shortBy` with an 'h' in backend
`direction`	String (default "0")	Sort order: `asc` or `desc`
`status`	String (default "ACTIVE")	Account status: `ACTIVE` or `INACTIVE`
`search`	String (optional)	Matches patron name or email address

Frontend Function: `memberService.getMembers({ page, size, sortBy, direction, search, status })` in `src/api/memberService.js`

UI Component: Powers status tabs and data table in `MembersPage.jsx` .

`GET` **`/api/members/{id}`** `MEMBERRESPONSE`

Purpose: Retrieves specific patron account details by member ID.

Frontend Function: `memberService.getMemberById(id)`

`POST` **`/api/members`** `STATUS: 201 CREATED`

Purpose: Registers a new patron account with credentials.

Request Body (MemberRequest):

```
{
  "name": "Ankita Zade",
  "email": "ankita@example.com",
  "phone": "8080976977",
  "password": "SecurePassword123"
}
```

Security Note: Password must be between 8 and 72 characters. The backend hashes passwords using BCrypt prior to database persistence.

Frontend Function: `memberService.createMember(memberData)` in `MemberFormModal.jsx` .

`PUT` **`/api/members/{id}`** `STATUS: 200 OK`

Purpose: Updates a patron's full name and contact phone number. Email address remains immutable.

Frontend Function: `memberService.updateMember(id, memberData)` in `MemberFormModal.jsx` .

`DELETE` **`/api/members/{id}`** `STATUS: 200 OK`

Purpose: Soft-deactivates a patron account by setting `status = INACTIVE` .

Frontend Function: `memberService.deactivateMember(id)` in `DeactivateMemberModal.jsx` .

---

5. Dashboard Real-Time Metric Aggregation

Because dedicated report endpoints (e.g. `/api/reports/*`) are not yet implemented in the backend, the Admin Dashboard calculates real-time metrics dynamically by executing parallel asynchronous requests via `Promise.all`:

```
const [booksData, activeMembersData, inactiveMembersData] = await Promise.all([
  bookService.getBooks({ page: 0, size: 8, sortBy: 'bookId', direction: 'desc' }),
  memberService.getMembers({ page: 0, size: 8, status: 'ACTIVE' }),
  memberService.getMembers({ page: 0, size: 1, status: 'INACTIVE' })
]);
```

Computed Metrics on Dashboard:

- **Total Book Titles:** Derived from `booksData.totalElements`.
- **Total Physical Stock:** Sum of `totalCopies` across catalog volumes.
- **Available Inventory:** Sum of shelf-available copies (`availableCopies`).
- **Circulating / Borrowed:** Calculated as `Total Physical Stock - Available Inventory`.
- **Inventory Health Ratio:** `(Available Copies / Total Copies) * 100%` rendered in dual-color progress bar.
- **Active vs. Inactive Accounts:** Extracted from `activeMembersData.totalElements` and `inactiveMembersData.totalElements`.

6. Summary API Mapping Reference Table

ACTION	METHOD & PATH	SERVICE FUNCTION	REACT COMPONENT	BACKEND CONTROLLER
Dashboard KPIs	`GET /api/books` & `GET /api/members`	`bookService.getBooks` & `memberService.getMembers`	`DashboardPage.jsx`	`BookController` & `MemberController`
List / Search Books	`GET /api/books`	`bookService.getBooks()`	`BooksPage.jsx`	`BookController.getAllBooks`
Add New Book	`POST /api/books`	`bookService.createBook()`	`BookFormModal.jsx`	`BookController.createBook`
Update Book	`PUT /api/books/{id}`	`bookService.updateBook()`	`BookFormModal.jsx`	`BookController.updateBook`
Delete Book	`DELETE /api/books/{id}`	`bookService.deleteBook()`	`DeleteBookModal.jsx`	`BookController.deleteBook`
List / Filter Members	`GET /api/members`	`memberService.getMembers()`	`MembersPage.jsx`	`MemberController.getAllMembers`
Register Member	`POST /api/members`	`memberService.createMember()`	`MemberFormModal.jsx`	`MemberController.createMember`
Update Member Profile	`PUT /api/members/{id}`	`memberService.updateMember()`	`MemberFormModal.jsx`	`MemberController.updateMember`
Deactivate Account	`DELETE /api/members/{id}`	`memberService.deactivateMember()`	`DeactivateMemberModal.jsx`	`MemberController.deleteMember`